

Модуль 7. Аутентификация, авторизация, безопасность

Введение в модуль

ML-сервис, обученный на дорогих данных и развёрнутый на мощном железе, бесполезен, если любой желающий может отправить запрос и получить предсказание. Ещё хуже — если злоумышленник может загрузить вредоносные данные, извлечь модель или исчерпать квоту GPU-инференса.

Безопасность — это не «фича», которую можно добавить в конце. Это **системное свойство**, пронизывающее каждый слой: от хранения паролей в базе до заголовков HTTP-ответа. В этом модуле мы разберём:

- **Криптографию с нуля:** почему SHA-256 нельзя использовать для паролей, как устроены цифровые подписи и почему ECDSA быстрее RSA.
- **Аутентификацию:** как пользователь доказывает, что он — это он. Реализацию OAuth2 Password Flow и Client Credentials в FastAPI.
- **Авторизацию:** как определить, что именно разрешено аутентифицированному пользователю. RBAC и ABAC.
- **Защиту приложения:** CORS, CSRF, XSS, SQL-инъекции, rate limiting, TLS и управление секретами.

7.1. Криптографические основы

7.1.1. Хеширование vs шифрование

Начинающие часто путают эти два понятия. Различие фундаментально.

Шифрование — обратимое преобразование. Есть ключ шифрования и ключ дешифрования (или один и тот же). Цель: скрыть содержимое от посторонних, но сохранить возможность восстановления для легитимного получателя.

$$\text{plaintext} \xrightarrow{\text{encrypt}(\text{key})} \text{ciphertext} \xrightarrow{\text{decrypt}(\text{key})} \text{plaintext}$$

Хеширование — необратимое преобразование. Функция H отображает вход произвольной длины в выход фиксированной длины (дайджест). Восстановление входа из выхода должно быть вычислительно невозможно.

$$\text{input} \xrightarrow{H} \text{digest}$$

Свойство	Шифрование	Хеширование
Обратимость	Да	Нет
Ключ	Требуется	Не требуется
Цель	Конфиденциальность	Целостность, проверка, хранение
Пример	AES-256-GCM	SHA-256, bcrypt

Пароли **хешируются**, а не шифруются. Если база данных скомпрометирована, злоумышленник не должен иметь возможности восстановить исходные пароли.

7.1.2. Хеширование паролей: почему не MD5/SHA-256

MD5 и SHA-256 — это **криптографические хеш-функции общего назначения**. Они спроектированы быть быстрыми. SHA-256 на современном GPU обрабатывает миллиарды паролей в секунду. Если злоумышленник украл хеши, он может перебрать миллиарды вариантов за минуты.

Пароли требуют **замедленных (deliberately slow)** хеш-функций — функций вывода ключа (Key Derivation Functions, KDF). Они должны быть:

- **Адаптивными:** параметр сложности можно увеличить по мере роста мощности железа.
- **Солеными:** каждый пароль хешируется с уникальной случайной «солью», чтобы одинаковые пароли давали разные хеши и чтобы нельзя было использовать заранее вычисленные таблицы (rainbow tables).

Соль (Salt)

$$\text{hash} = H(\text{password} \parallel \text{salt})$$

Соль — случайная строка длиной ≥ 16 байт, хранящаяся рядом с хешем. Без соли:

Пользователь	Пароль	MD5
Alice	password123	482c811da5d5b4bc6d497ffa98491e38
Bob	password123	482c811da5d5b4bc6d497ffa98491e38

Злоумышленник видит совпадение и взламывает оба аккаунта сразу. С солью хеши различаются даже при одинаковом пароле.

bcrypt

Разработан в 1999 году, основан на шифре Blowfish. Ключевая особенность — параметр **cost factor** (4–31, по умолчанию 12). Каждое увеличение cost удваивает время хеширования.

```
In [1]: from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Хеширование
hashed = pwd_context.hash("mysecretpassword")
# $2b$12$... — 2b = bcrypt версия, 12 = cost factor

# Проверка
is_valid = pwd_context.verify("mysecretpassword", hashed)
```

Почему bcrypt хорош: он использует операции, неэффективные на GPU (доступ к памяти с непредсказуемым паттерном), что замедляет массовый перебор.

Argon2

Победитель **Password Hashing Competition** (2015). Три параметра настройки:

- **memory:** сколько памяти использовать (например, 64 МБ).
- **iterations:** число проходов.
- **parallelism:** число потоков.

Argon2 специально спроектирован быть **memory-hard**: чтобы ускорить перебор на GPU/ASIC, нужно линейно увеличивать память, что дорого.

```
In [2]: pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")
hashed = pwd_context.hash("mysecretpassword")
```

Рекомендация OWASP (2023): Argon2id — предпочтительный алгоритм для новых систем. bcrypt — приемлемая альтернатива. scrypt — если нужна memory-hard функция с простой реализацией. MD5, SHA-1, SHA-256 для паролей — категорически запрещены.

7.1.3. JWT: структура и принципы

JSON Web Token (JWT, RFC 7519) — компактный способ передавать утверждения (claims) между сторонами в виде подписанного JSON.

Структура: `header.payload.signature`, каждая часть кодируется в **Base64Url** (без padding `=` и с заменой `+` -> `-`, `/` -> `_`).

Header

```
In [3]: {
  "alg": "HS256",
  "typ": "JWT"
}
```

`alg` — алгоритм подписи. `typ` — тип токена (обычно JWT).

Payload (Claims)

```
In [4]: {
  "sub": "1234567890",
  "name": "Alice",
  "iat": 1516239022,
  "exp": 1516242622,
  "scope": "predict:read predict:write"
}
```

Стандартные claims:

- `sub` (subject): идентификатор пользователя.
- `iat` (issued at): время выпуска.
- `exp` (expiration): время истечения.
- `nbf` (not before): не действителен до.
- `iss` (issuer): издатель.
- `aud` (audience): получатели.
- `jti` (JWT ID): уникальный идентификатор токена (для отзыва).

Signature

Для HS256:

$$\text{Signature} = \text{HMAC}_{\text{SHA256}}(\text{Base64Url}(\text{header}) + "." + \text{Base64Url}(\text{payload}), \text{secret})$$

Для RS256:

$$\text{Signature} = \text{RSA}_{\text{sign}}(\text{SHA256}(\text{Base64Url}(\text{header}) + "." + \text{Base64Url}(\text{payload})), \text{private}_{\text{key}})$$

Проверка: получатель вычисляет хеш от `header.payload` и сверяет подпись с помощью ключа.

Stateless vs Stateful сессии

Подход	Stateful (Session ID)	Stateless (JWT)
Хранение	Сессия на сервере (Redis, БД)	Вся информация в токене
Проверка	Запрос к хранилищу сессий	Проверка подписи локально
Отзыв	Мгновенный (удалить сессию)	Сложный (blacklist, короткий TTL)
Масштабируемость	Требует shared storage	Любой сервер проверяет сам
Размер	16–32 байта (cookie)	Сотни байт–килобайты (заголовок)

Вывод: JWT удобен для распределённых систем (микросервисы), но не лишён недостатков. Access token должен быть **короткоживущим** (5–15 минут), чтобы минимизировать окно компрометации.

7.1.4. Алгоритмы подписи JWT: HS256, RS256, ES256

HS256 (HMAC-SHA256) — симметричный. Один и тот же `secret` используется и для подписи, и для проверки.

- **Плюс:** простота, скорость.
- **Минус:** все сервисы, проверяющие токен, знают секрет и могут подписывать. Компрометация секрета = компрометация всей системы.

RS256 (RSA + SHA-256) — асимметричный. Приватный ключ подписывает, публичный проверяет.

- **Плюс:** публичный ключ можно распространять безопасно. Микросервисы проверяют токен, не имея возможности подделать.
- **Минус:** RSA медленнее HMAC, подпись и ключи большие (2048+ бит).

ES256 (ECDSA + SHA-256 + curve P-256) — асимметричный на эллиптических кривых.

- **Плюс:** подпись в 2 раза короче RSA при эквивалентной стойкости (64 байта vs 256 байт). Быстрее.
- **Минус:** математика сложнее, требует качественной генерации случайных чисел (при повторном *k* возможна утечка приватного ключа).

7.1.5. Математическая подоплека: асимметричная криптография

RSA

Основана на **трудности факторизации** больших чисел.

1. Выбираются два больших простых числа p и q .
2. Вычисляется $n = p \cdot q$ (модуль, публичный).
3. Вычисляется $\varphi(n) = (p - 1)(q - 1)$.
4. Выбирается публичная экспонента e (обычно 65537).
5. Вычисляется приватная экспонента $d = e^{-1} \bmod \varphi(n)$.

Публичный ключ: (n, e) . **Приватный ключ:** (n, d) .

Подпись сообщения m :

$$s = m^d \bmod n$$

Проверка подписи:

$$m' = s^e \bmod n \stackrel{?}{=} m$$

Корректность следует из теоремы Эйлера: $(m^d)^e = m^{de} \equiv m \pmod{n}$.

Безопасность: вычислить d из (n, e) требует факторизации n . Для $n \sim 2048$ бит это вычислительно невозможно на классических компьютерах.

ECDSA (Elliptic Curve Digital Signature Algorithm)

Основан на **трудности дискретного логарифма** на эллиптических кривых.

Эллиптическая кривая — множество точек (x, y) , удовлетворяющих уравнению:

$$y^2 = x^3 + ax + b \pmod{p}$$

вместе с «точкой на бесконечности» $\mathcal{O}$.

Групповая операция — **сложение точек**: если P и Q — точки на кривой, то $P + Q$ — тоже точка на кривой (геометрически: провести прямую через P и Q , найти третье пересечение с кривой, отразить относительно оси x).

Умножение точки на скаляр:

$$k \cdot P = \underbrace{P + P + \dots + P}_{k \text{ раз}}$$

Вычисляется за $O(\log k)$ методом удвоения-сложения.

Генерация ключей:

- Приватный ключ: случайное число $d \in [1, n - 1]$, где n — порядок группы.
- Публичный ключ: точка $Q = d \cdot G$, где G — фиксированная генераторная точка.

Подпись:

1. Выбирается случайное k (уникальное для каждой подписи!).
2. $R = k \cdot G = (x_R, y_R)$, $r = x_R \bmod n$.
3. $s = k^{-1}(H(m) + d \cdot r) \bmod n$.
4. Подпись: (r, s) .

Проверка: используя Q (публичный ключ), проверяется равенство, связывающее r , s , $H(m)$ и Q .

Почему это безопасно: вычислить d по $Q = d \cdot G$ — это **дискретный логарифм** на эллиптической кривой (ECDLP). Для хорошо выбранной кривой (например, NIST P-256, Curve25519) эта задача вычислительно неразрешима.

Почему ECDSA быстрее RSA: подпись ECDSA P-256 — 64 байта. Подпись RSA-2048 — 256 байт. Операции на кривых требуют меньших чисел (~ 256 бит vs ~ 2048 бит).

Односторонние функции

Криптография опирается на **односторонние функции с потайным ходом (trapdoor one-way functions)**:

$$f : X \rightarrow Y$$

- Легко вычислить $y = f(x)$ для любого x .
- Трудно найти x по y .
- При наличии **потайного хода** (secret) обращение становится легким.

Для хеширования паролей потайного хода нет — функция необратима в принципе. Для RSA потайной ход — знание факторизации $n = p \cdot q$.

7.2. Реализация аутентификации в FastAPI

7.2.1. OAuth2: фреймворк авторизации

OAuth 2.0 (RFC 6749) — это **фреймворк авторизации**, а не протокол аутентификации. Он описывает, как приложение (client) получает **делегированный доступ** к ресурсам пользователя от сервера авторизации (authorization server).

Flows (Grant Types):

Flow	Когда использовать	Участники
Authorization Code	Серверные веб-приложения (Google Login)	User, Browser, Client App, Auth Server
Authorization Code + PKCE	SPA, мобильные приложения (без client_secret)	Тот же, но с proof key
Password (устарел в OAuth 2.1)	Доверенные first-party приложения	User передаёт логин/пароль напрямую
Client Credentials	Machine-to-machine (сервисы)	Client App, Auth Server
Device Code	Устройства без браузера (TV, IoT)	User авторизуется на другом устройстве

Для ML-API чаще всего используются:

- **Password Flow** — для внутренних пользователей (если нет SSO).
- **Client Credentials** — для микросервисов и внешних API-клиентов.

7.2.2. Password Flow на практике

FastAPI предоставляет встроенные инструменты для OAuth2 Password Flow.

```
In [5]: from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

app = FastAPI()

# URL, по которому клиент запрашивает токен
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    # form_data.username и form_data.password
    user = await authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token(data={"sub": user.username})
    return {"access_token": access_token, "token_type": "bearer"}
```



```
@app.get("/predictions")
async def read_predictions(token: str = Depends(oauth2_scheme)):
    # FastAPI извлекает токен из заголовка Authorization: Bearer <token>
    payload = verify_token(token)
    return {"predictions": []}
```

Что происходит:

1. Клиент отправляет `POST /token` с `username` и `password` (form-data).
2. Сервер проверяет учётные данные (хеш пароля в БД).
3. Генерирует JWT access token.
4. Клиент использует токен в заголовке `Authorization: Bearer eyJhbGciOiJIUzI1NiIs...`.
5. FastAPI через `OAuth2PasswordBearer` автоматически извлекает токен и передаёт в endpoint.

OAuth2PasswordRequestForm — это Pydantic-модель, которая ожидает форму с полями:

- `username` (обязательно)
- `password` (обязательно)
- `scope` (опционально)
- `client_id`, `client_secret` (опционально, для клиентской аутентификации)

7.2.3. Регистрация, верификация email, сброс пароля

Регистрация

```
In [6]: from pydantic import BaseModel, EmailStr, field_validator

class UserCreate(BaseModel):
    username: str
    email: EmailStr
    password: str

    @field_validator('password')
    @classmethod
    def validate_password(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError('Password must be at least 8 characters')
        return v

@app.post("/register")
async def register(user_data: UserCreate):
    # Проверка уникальности
```

```

existing = await get_user_by_email(user_data.email)
if existing:
    raise HTTPException(status_code=400, detail="Email already registered")

hashed_password = pwd_context.hash(user_data.password)

# Создание пользователя (is_active=False до верификации)
user = await create_user(
    username=user_data.username,
    email=user_data.email,
    hashed_password=hashed_password,
    is_active=False
)

# Отправка письма с токеном верификации
token = create_verification_token(user.id)
await send_email(user.email, "Verify your email", f"Click: /verify?token={token}")

return {"message": "Check your email"}

```

Верификация email

```

In [7]: @app.get("/verify")
        async def verify_email(token: str):
            payload = verify_token(token, token_type="verify")
            user_id = payload.get("sub")

            user = await get_user_by_id(int(user_id))
            if not user:
                raise HTTPException(status_code=404)

            await activate_user(user.id)
            return {"message": "Email verified"}

```

Токен верификации — отдельный JWT с коротким сроком жизни (1–24 часа) и типом `verify`, чтобы его нельзя было использовать как access token.

Сброс пароля

1. Пользователь запрашивает сброс: `POST /forgot-password` с email.
2. Сервер генерирует одноразовый токен сброса (JWT или криптографически случайная строка).
3. Отправляет ссылку на email.
4. Пользователь переходит по ссылке, вводит новый пароль.

5. Сервер проверяет токен, хеширует новый пароль, обновляет в БД.

```
In [8]: @app.post("/reset-password")
async def reset_password(token: str, new_password: str):
    payload = verify_token(token, token_type="reset")
    user_id = int(payload["sub"])

    hashed = pwd_context.hash(new_password)
    await update_password(user_id, hashed)

    # Инвалидация всех существующих сессий/токенов (опционально)
    await invalidate_user_tokens(user_id)

    return {"message": "Password updated"}
```

7.2.4. Access Token и Refresh Token

Проблема: если access token живёт долго (например, 24 часа) и украден — злоумышленник 24 часа имеет доступ. Если access token короткий (5 минут) — пользователю приходится вводить пароль каждые 5 минут.

Решение: два токена.

Токен	Срок жизни	Хранение	Назначение
Access Token	5–15 минут	Память приложения (JS-переменная)	Доступ к API
Refresh Token	7–30 дней	httpOnly cookie или secure storage	Получение нового access token

Почему refresh token в httpOnly cookie?

- `httpOnly` запрещает JavaScript читать cookie — защита от XSS.
- `secure` — отправляется только по HTTPS.
- `SameSite=strict` — защита от CSRF.

Ротация Refresh Token

При каждом обмене refresh token на новую пару (access + refresh):

1. Старый refresh token **аннулируется**.
2. Выдаётся **новый** refresh token.

3. Если кто-то попытается использовать старый refresh token — это признак кражи. Все токены пользователя аннулируются.

```
In [9]: @app.post("/refresh")
async def refresh_token(refresh_token: str = Cookie(None)):
    if not refresh_token:
        raise HTTPException(status_code=401, detail="No refresh token")

    payload = verify_token(refresh_token, token_type="refresh")
    user_id = int(payload["sub"])
    token_jti = payload["jti"] # уникальный ID токена

    # Проверяем, не отозван ли токен
    if await is_token_revoked(token_jti):
        # Возможная кража: аннулируем все токены пользователя
        await revoke_all_user_tokens(user_id)
        raise HTTPException(status_code=401, detail="Token revoked")

    # Аннулируем старый refresh token
    await revoke_token(token_jti)

    # Генерируем новую пару
    new_access = create_access_token({"sub": user_id})
    new_refresh = create_refresh_token({"sub": user_id, "jti": generate_jti()})

    response = JSONResponse({"access_token": new_access})
    response.set_cookie(
        key="refresh_token",
        value=new_refresh,
        httponly=True,
        secure=True,
        samesite="strict",
        max_age=7 * 24 * 3600,
    )
    return response
```

7.3. Авторизация и RBAC

7.3.1. Аутентификация ≠ Авторизация

- **Аутентификация** — установление личности. «Ты — кто ты есть?»
- **Авторизация** — определение прав. «Что тебе разрешено делать?»

Аутентификация без авторизации бессмысленна: любой залогиненный пользователь может удалить чужую модель.

7.3.2. RBAC: Role-Based Access Control

RBAC — права назначаются ролям, роли назначаются пользователям.

In [10]: Пользователь <--> Роль <--> Разрешение <--> Ресурс

Пример для ML-платформы:

Роль	Разрешения
admin	user:* , model:* , prediction:*
data_scientist	model:read , model:write , prediction:read
analyst	prediction:read , dashboard:read
api_client	prediction:create

Реализация через зависимости FastAPI:

```
In [11]: from enum import Enum

class Role(str, Enum):
    ADMIN = "admin"
    DATA_SCIENTIST = "data_scientist"
    ANALYST = "analyst"
    API_CLIENT = "api_client"

ROLE_PERMISSIONS = {
    Role.ADMIN: {"user:*", "model:*", "prediction:*"},
    Role.DATA_SCIENTIST: {"model:read", "model:write", "prediction:read"},
    Role.ANALYST: {"prediction:read", "dashboard:read"},
    Role.API_CLIENT: {"prediction:create"},
}

class User(BaseModel):
    id: int
    username: str
    role: Role

async def get_current_user(token: str = Depends(oauth2_scheme)) -> User:
```

```

payload = verify_token(token)
user = await get_user_by_username(payload["sub"])
return user

class PermissionChecker:
    def __init__(self, required_permissions: list[str]):
        self.required_permissions = required_permissions

    def __call__(self, user: User = Depends(get_current_user)):
        user_perms = ROLE_PERMISSIONS.get(user.role, set())

        # Проверка wildcard: "model:*" даёт доступ к "model:read"
        for required in self.required_permissions:
            if required in user_perms:
                continue

            resource, action = required.split(":")
            if f"{resource}:*" in user_perms:
                continue

            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Permission {required} required"
            )

        return user

@app.post("/models/", dependencies=[Depends(PermissionChecker(["model:write"]))])
async def create_model(user: User = Depends(get_current_user)):
    return {"message": "Model created"}

```

7.3.3. ABAC: Attribute-Based Access Control

RBAC грубый: роль определяет права статически. ABAC позволяет динамические правила на основе **атрибутов**:

- Пользователя: отдел, уровень допуска, верификация email.
- Ресурса: владелец модели, статус (draft/published).
- Действия: время суток, IP-адрес.
- Окружения: production vs staging.

Пример ABAC-политики:

```

In [12]: def can_delete_model(user: User, model: Model) -> bool:
        # Админ может всё
        if user.role == Role.ADMIN:
            return True
        # Владелец может удалять свои черновики

```

```

if model.owner_id == user.id and model.status == "draft":
    return True
# Ночью (00:00-06:00) никто не удаляет
if datetime.now().hour < 6:
    return False
return False

```

ABAC гибче, но сложнее в аудите и тестировании. Для большинства ML-сервисов достаточно RBAC с небольшими ABAC-вкраплениями (например, проверка владельца ресурса).

7.3.4. Декораторы и зависимости для проверки прав

Можно создать специализированные зависимости:

```

In [13]: async def require_admin(user: User = Depends(get_current_user)):
        if user.role != Role.ADMIN:
            raise HTTPException(status_code=403, detail="Admin required")
        return user

@app.delete("/users/{user_id}")
async def delete_user(user_id: int, admin: User = Depends(require_admin)):
    await delete_user_from_db(user_id)
    return {"message": "User deleted"}

```

Или комбинировать:

```

In [14]: @app.get("/admin/stats")
        async def admin_stats(
            user: User = Depends(get_current_user),
            _: None = Depends(PermissionChecker(["dashboard:read"]))
        ):
            return {"stats": "secret data"}

```

7.4. Безопасность приложения

7.4.1. CORS (Cross-Origin Resource Sharing)

Same-Origin Policy браузера запрещает JavaScript на `https://evil.com` делать запросы к `https://bank.com`. Но API должно быть доступно фронтенду на другом домене.

CORS — механизм, при котором сервер **явно разрешает** определённым источникам (origins) обращаться к ресурсам.

```
In [15]: from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com"], # конкретные домены, не "*"
    allow_credentials=True, # разрешить cookies/authorization headers
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)
```

Preflight запрос:

Для «непростых» запросов (методы отличные от GET/HEAD/POST, или кастомные заголовки) браузер сначала отправляет **OPTIONS**-запрос:

```
In [16]: OPTIONS /api/predict HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, Authorization
```

Сервер должен ответить:

```
In [17]: HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: POST
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Max-Age: 86400
```

Опасность: `allow_origins=["*"]` с `allow_credentials=True` — уязвимость. Любой сайт может делать аутентифицированные запросы от имени пользователя.

7.4.2. CSRF, XSS, SQL Injection

CSRF (Cross-Site Request Forgery)

Злоумышленник заставляет браузер аутентифицированного пользователя выполнить нежелательное действие.

```
In [18]: Пользователь залогинен в bank.com (cookie с session_id).
Заходит на evil.com.
```



```
evil.com содержит:   
Браузер отправляет запрос с cookie bank.com. Деньги уходят.
```

Защита:

- **SameSite cookies:** `SameSite=strict` или `SameSite=lax` (cookie не отправляется при cross-site запросах).
- **CSRF-токены:** сервер выдаёт уникальный токен, который должен быть в каждом mutating-запросе. Для API с JWT в заголовке CSRF неактуален (злоумышленник не может подделывать заголовки с чужого домена из-за Same-Origin Policy).

XSS (Cross-Site Scripting)

Злоумышленник внедряет JavaScript в страницу, которую видит жертва.

Stored XSS: внедрение через сохранённые данные (комментарий с `<script>`). **Reflected XSS:** внедрение через URL-параметры.

Защита:

- **Экранирование вывода:** FastAPI + Pydantic по умолчанию экранируют JSON. Но если вы возвращаете HTML — используйте шаблонизаторы с автоэкранированием (Jinja2).
- **Content-Security-Policy (CSP):** заголовок, запрещающий выполнение inline-скриптов.

```
In [19]: @app.middleware("http")  
         async def add_csp(request, call_next):  
             response = await call_next(request)  
             response.headers["Content-Security-Policy"] = "default-src 'self'; script-src 's  
             elf'"  
             return response
```

SQL Injection

```
In [20]: # УЯЗВИМОСТЬ:  
         query = f"SELECT * FROM users WHERE name = '{user_input}'"  
         # user_input = "'; DROP TABLE users; --"
```

SQLAlchemy ORM защищает от SQL-инъекций через **параметризованные запросы**:

```
In [21]: # БЕЗОПАСНО:  
         result = await session.execute(select(User).where(User.name == user_input))
```

Но raw SQL требует осторожности:

```
In [22]: # УЯЗВИМО, если user_input не проверен:
result = await session.execute(text(f"SELECT * FROM users WHERE name = '{user_input}'"))

# БЕЗОПАСНО:
result = await session.execute(
    text("SELECT * FROM users WHERE name = :name"),
    {"name": user_input}
)
```

7.4.3. Rate Limiting

Ограничение числа запросов от одного клиента для защиты от:

- Brute-force атак на аутентификацию.
- Истощения квоты ML-инференса.
- DDoS.

Token Bucket (ведро с токенами)

Абстракция: ведро вмещает B токенов. Токены поступают со скоростью r токенов/сек. Запрос стоит 1 токен. Если токенов нет — запрос отклоняется.

Параметры:

- **Capacity (B):** максимальный размер всплеска (burst).
- **Rate (r):** устойчивая скорость.

Математически, число токенов в момент t :

$$tokens(t) = \min(B, tokens(t_0) + r \cdot (t - t_0) - consumed)$$

Реализация на Redis:

```
In [23]: import asyncio
from redis.asyncio import Redis

redis = Redis()

async def token_bucket_check(key: str, capacity: int, rate: float) -> bool:
    """Возвращает True, если запрос разрешён."""
    pipe = redis.pipeline()
```

```

now = time.time()

# Получаем текущее состояние
pipe.hmget(key, "tokens", "last_update")
result = await pipe.execute()

tokens = float(result[0][0]) if result[0][0] else capacity
last_update = float(result[0][1]) if result[0][1] else now

# Пополняем токены
elapsed = now - last_update
tokens = min(capacity, tokens + elapsed * rate)

if tokens >= 1:
    tokens -= 1
    await redis.hset(key, mapping={"tokens": tokens, "last_update": now})
    return True
else:
    await redis.hset(key, "last_update", now)
    return False

```

Sliding Window (скользящее окно)

Более точный, чем token bucket. Считает запросы за последние T секунд.

```

In [24]: async def sliding_window_check(key: str, window: int, limit: int) -> bool:
    now = time.time()
    window_start = now - window

    pipe = redis.pipeline()
    # Удаляем устаревшие записи
    pipe.zremrangebyscore(key, 0, window_start)
    # Считаем текущие
    pipe.zcard(key)
    # Добавляем текущий запрос
    pipe.zadd(key, {str(now): now})
    # Устанавливаем TTL на ключ
    pipe.expire(key, window + 1)

    _, current_count, _, _ = await pipe.execute()

    return current_count < limit

```

Сравнение:

Алгоритм	Burst	Точность	Сложность
Token Bucket	Допускает	Средняя	Низкая
Sliding Window	Сглаживает	Высокая	Средняя
Fixed Window	Резкий сброс в границе окна	Низкая	Низкая

7.4.4. HTTPS/TLS

TLS (Transport Layer Security) — криптографический протокол, обеспечивающий:

- **Конфиденциальность:** шифрование данных.
- **Целостность:** защита от подмены.
- **Аутентификацию сервера:** клиент уверен, что общается с настоящим сервером.

TLS Handshake (упрощённо)

```
In [25]:
Клиент                                Сервер
|                                     |
|----- ClientHello ----->| (поддерживаемые cipher suites, случайное число)
|                                     |
|<----- ServerHello -----| (выбранный cipher, сертификат, случайное число)
|<----- Certificate -----| (X.509 цепочка)
|<----- ServerHelloDone -----|
|                                     |
|----- ClientKeyExchange ----->| (премастер-секрет, зашифрованный публичным ключом)
|----- [ChangeCipherSpec] ----->|
|----- Finished ----->| (проверка handshake)
|                                     |
|<----- [ChangeCipherSpec] -----|
|<----- Finished -----|
|                                     |
[Зашифрованное соединение]
```

Сертификаты X.509: иерархия доверия. Сервер предъявляет сертификат, подписанный **Certificate Authority (CA)**. Браузер проверяет подпись CA (у которого есть публичный ключ в системе), убеждается, что домен совпадает, и что сертификат не просрочен.

Let's Encrypt: бесплатный CA, автоматизирующий выдачу сертификатов через протокол ACME. Сертификаты действительны 90 дней и обновляются автоматически (например, через `certbot`).

Терминация SSL:

- **На балансировщике** (Nginx, Traefik, AWS ALB): балансировщик расшифровывает TLS, общается с FastAPI по обычному HTTP внутри приватной сети. Проще, но трафик внутри сети нешифрован.
- **На приложении** (Uvicorn с TLS): FastAPI сам расшифровывает TLS. Более безопасно, но сложнее в управлении сертификатами.

```
In [26]: # Uvicorn с TLS
         uvicorn main:app --ssl-keyfile=./key.pem --ssl-certfile=./cert.pem
```

7.4.5. Хранение секретов

Секреты — пароли к БД, API-ключи, приватные ключи для JWT — не должны быть в коде.

Иерархия подходов:

Уровень	Подход	Пример
1	Жёстко в коде	<code>SECRET = "abc123"</code> — категорически запрещено
2	Переменные окружения	<code>os.environ["SECRET_KEY"]</code> — минимум
3	Файлы секретов	Docker secrets, Kubernetes secrets
4	Vault	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault

Pydantic Settings:

```
In [27]: from pydantic_settings import BaseSettings

         class Settings(BaseSettings):
             database_url: str
             secret_key: str
             algorithm: str = "HS256"
             access_token_expire_minutes: int = 15

             class Config:
                 env_file = ".env"
                 env_file_encoding = "utf-8"

         settings = Settings()
```

HashiCorp Vault: динамические секреты. Вместо статического пароля к БД приложение запрашивает у Vault временные учётные данные с TTL. Даже при компрометации

злоумышленник получает доступ только на ограниченное время.

Итог модуля 7

Концепция	Суть	Практическое применение
Хеширование паролей	Одностороннее замедленное преобразование	Argon2id или bcrypt через <code>passlib</code>
JWT	Подписанные claims для stateless аутентификации	Access token (короткий) + Refresh token (долгий, httpOnly)
Асимметричная криптография	RSA/ECDSA: подпись приватным, проверка публичным	RS256/ES256 для распределённых микросервисов
OAuth2 Password Flow	Обмен логина/пароля на токен	Внутренние API с доверенными клиентами
Client Credentials	Токен для сервис-сервис	ML inference между микросервисами
RBAC	Роли -> разрешения -> ресурсы	Ограничение доступа к админке, моделям, предсказаниям
ABAC	Права на основе атрибутов	Владелец модели может редактировать, другие — нет
CORS	Контроль cross-origin доступа	Разрешить фронтенду, запретить всем остальным
Rate Limiting	Ограничение числа запросов	Token bucket для API, sliding window для аутентификации
TLS	Шифрование транспорта	Let's Encrypt, терминатор на балансировщике
Хранение секретов	Изоляция чувствительных данных	Vault, env vars, никогда в коде

В **Модуле 8** мы разберём тестирование асинхронных приложений: как писать unit-тесты для корутин, интеграционные тесты с тестовой БД, нагрузочное тестирование и профилирование.